Ryerson University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed and Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	W2026

Instructor:	Dr. Muhammad Jaseemuddin

Assignment/Lab Number:	3
Assignment/Lab Title:	RabbitMQ

Submission Date:	Mar 13, 2026
Due Date:	Mar 13, 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Pathirana	Chanuth	501107354	01	C.P

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

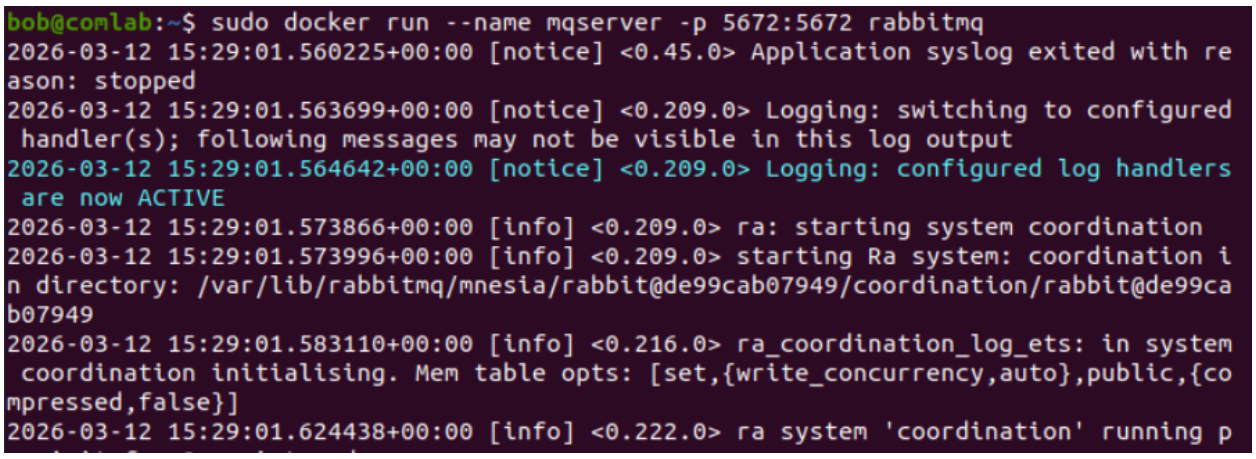
Introduction

The objective of this lab is to build on Lab 2 by developing three Python programs that facilitate communication between the rovers, deminers, and ground control using gRPC and RabbitMQ. Unlike the previous lab, where rovers performed both exploration and digging, their role is now limited to identifying mines and requesting the gRPC server to retrieve mine serial numbers. Once a mine is detected, the rover publishes a demining task via RabbitMQ, which the newly introduced deminers consume. The deminers retrieve mine coordinates, IDs, and serial numbers from the queue and proceed to disarm mines sequentially if they are not busy. Upon successful disarming, the deminers publish the mine's PIN over another RabbitMQ channel, where ground control acts as the consumer. This lab aims to simulate real-world asynchronous communication, leveraging message queuing and remote procedure calls to coordinate autonomous agents in a distributed system effectively.

Implementation

Part 1: Create the RabbitMQ server

The first step of the lab was to create and run a RabbitMQ server that would handle the message queues used for communication between rovers, deminers, and ground control. RabbitMQ was executed inside a Docker container to simplify deployment and ensure a consistent environment. To run the server, I had to use the command: sudo docker run --name mqserver -p 5672:5672 rabbitmq. To use the server for a new session, I had to stop the mqserver by using the command: sudo docker stop mqserver. After that, I had to remove the mqserver with the command: sudo docker rm mqserver so that I could run the docker run command in a new session. I had to use the sudo command to get permission to run Docker and had to type in the password letbobin for the username, bob.



Part 2: Create the Ground Control

Ground Control manages communication between the rovers and the mine-defusing system. It utilizes the map files from the previous two labs and processes rover command files, excluding the digging command. The server is built based on the .porto files from Lab 2 but only implements three methods: GetMap, GetCommands, and GetMineSerial. It acts as a gRPC server that provides the services to rover clients, and it also acts as the RabbitMQ consumer that receives notifications whenever the mines are disarmed. The read_map() method is used to retrieve the 2D map file from Lab 1 and 2. Rover command files are used by the load_rover_commands() method. The mine serial number file is retrieved throught the load_mines() method. The server subscribes to the defused-mines channel queue and listens for messages with the start_defused_consumer() method.

```
import grpc
from concurrent import futures
import time
import copy
import hashlib
import threading

import rover_pb2
import rover_pb2_grpc

import pika  # RabbitMQ client

# ---------- MAP / MINES REUSE FROM LAB 1 ----------

def read_map(filename="map1.txt"):
    with open(filename, "r") as f:
        first_line = f.readline().strip().split()
        rows, cols = int(first_line[0]), int(first_line[1])
        land = []
        for _ in range(rows):
            land.append([int(x) for x in f.readline().strip().split()])
    return rows, cols, land

ROWS, COLS, BASE_MAP = read_map("map1.txt")

def load_mines(filename="mines.txt"):
    mines = []
    with open(filename, "r") as f:
        for line in f:
            serial = line.strip()
            if serial:
                mines.append(serial)
    return mines

MINES = load_mines("mines.txt")
```

```
def get_mine_positions(land):
    positions = []
    for r in range(len(land)):
        for c in range(len(land[0])):
            if land[r][c] == 1:
                positions.append((r, c))
    return positions

MINE_POSITIONS = get_mine_positions(BASE_MAP)

MINE_SERIAL_MAP = {}
for idx, (r, c) in enumerate(MINE_POSITIONS):
    if idx < len(MINES):
        MINE_SERIAL_MAP[(r, c)] = MINES[idx]
    else:
        MINE_SERIAL_MAP[(r, c)] = f"EXTRA-{idx}"

# ---------- COMMANDS FOR EACH ROVER ----------

def load_rover_commands(rover_id: int) -> str:
    import random
    random.seed(rover_id)
    # In Lab 3, you *won't* use D on the rover side, but server can still
generate it
    commands = ["L", "R", "M", "D"]
    seq = "".join(random.choice(commands) for _ in range(20))
    return seq

# ---------- VERIFY PIN FUNCTION (still usable if needed) ----------

def verify_pin(pin, serial):
    temp_key = str(pin) + serial
    h = hashlib.sha256(temp_key.encode()).hexdigest()
    return h.startswith("000000")

# ---------- RABBITMQ CONSUMER FOR DEFUSED-MINES ----------

def defused_callback(ch, method, properties, body):
    pin = body.decode()
    print(f"[GROUND CONTROL] Defused mine PIN received: {pin}")

def start_defused_consumer():
    """
    Subscribe to the 'Defused-Mines' queue and print any received PINs.
    """
    connection = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
    channel = connection.channel()
    channel.queue_declare(queue="Defused-Mines")
```

```
    channel.basic_consume(
        queue="Defused-Mines",
        on_message_callback=defused_callback,
        auto_ack=True,
    )

    print("[GROUND CONTROL] Waiting for defused mines on 'Defused-Mines'
queue...")
    try:
        channel.start_consuming()
    except KeyboardInterrupt:
        channel.stop_consuming()
        connection.close()

# ---------- SERVICE IMPLEMENTATION ----------

class RoverControlServicer(rover_pb2_grpc.RoverControlServicer):
    def GetMap(self, request, context):
        land = copy.deepcopy(BASE_MAP)
        flat = []
        for r in range(ROWS):
            for c in range(COLS):
                flat.append(land[r][c])
        map_data = rover_pb2.MapData(
            rows=ROWS,
            cols=COLS,
            data=flat
        )
        return rover_pb2.MapResponse(map=map_data)

    def GetCommands(self, request, context):
        rover_id = request.rover_id
        sequence = load_rover_commands(rover_id)
        for ch in sequence:
            yield rover_pb2.Command(cmd=ch)

    def GetMineSerial(self, request, context):
        key = (request.row, request.col)
        serial = MINE_SERIAL_MAP.get(key, "")
        return rover_pb2.MineSerialResponse(serial=serial)

    # You may keep these or ignore them for Lab 3 if not needed
    def ReportExecutionStatus(self, request, context):
        rover_id = request.rover_id
        success = request.success
        msg = request.message
        print(f"[SERVER] Rover {rover_id} finished. Success={success},
msg={msg}")
        return rover_pb2.ExecutionStatusResponse(ack=True)
```

```
    def SubmitMinePin(self, request, context):
        rover_id = request.rover_id
        serial = request.serial
        pin = request.pin

        print(f"[SERVER] Rover {rover_id} submitted PIN {pin} for mine
{serial}")

        valid = verify_pin(pin, serial)

        if valid:
            print(f"[SERVER] PIN verified correct!")
        else:
            print(f"[SERVER] PIN invalid!")

        return rover_pb2.MinePinResponse(accepted=valid)

# ---------- SERVER BOOTSTRAP ----------

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    rover_pb2_grpc.add_RoverControlServicer_to_server(
        RoverControlServicer(), server
    )

    server.add_insecure_port("[::]:50051")
    server.start()
    print("RoverControl gRPC server running on port 50051...")

    # Start RabbitMQ consumer in a background thread
    consumer_thread = threading.Thread(
        target=start_defused_consumer,
        daemon=True
    )
    consumer_thread.start()

    try:
        while True:
            time.sleep(86400)
    except KeyboardInterrupt:
        server.stop(0)

if __name__ == "__main__":
    serve()
```

server.py

Part 3: Create the Rovers

The rover client (Part 3) implements a gRPC client that connects to the gRPC server running on port 50051 which accepts a CLI argument (rover ID 1-10) and performs sequential map exploration without threading, as specified. It reuses Lab 1/2 navigation logic but replaces local mine disarming with RabbitMQ task publishing. The rover explores the map sequentially by using the loop “for cmd_msg in cmd_stream:” which traverses through all the commands in order. When the rover moves to a cell containing 1, it is able to detect a mine in need of disarming the prints out a statement that reveals the location of the mine.

```
import grpc
import sys
import copy
import pika  # RabbitMQ
import json

import rover_pb2
import rover_pb2_grpc

# Directions and movement (from Lab 1)
DIRECTIONS = ["N", "E", "S", "W"]
MOVE = {
    "N": (-1, 0),
    "E": (0, 1),
    "S": (1, 0),
    "W": (0, -1),
}

def turn_left(d):
    return DIRECTIONS[(DIRECTIONS.index(d) - 1) % 4]

def turn_right(d):
    return DIRECTIONS[(DIRECTIONS.index(d) + 1) % 4]

def build_map_from_response(map_resp):
    rows = map_resp.map.rows
    cols = map_resp.map.cols
    flat = list(map_resp.map.data)
    land = []
    idx = 0
    for r in range(rows):
        row = []
        for c in range(cols):
            row.append(flat[idx])
            idx += 1
        land.append(row)
    return rows, cols, land

def publish_mine_task(rover_id, nx, ny, serial):
    """
```

```
    Publish mine coordinates, rover ID (as mine_id?), and serial to
Demine-Queue.
    """
    connection = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
    channel = connection.channel()
    channel.queue_declare(queue="Demine-Queue")

    task = {
        "row": nx,
        "col": ny,
        "mine_id": rover_id,  # Using rover_id as mine_id; adjust if needed
        "serial": serial
    }

    channel.basic_publish(
        exchange="",
        routing_key="Demine-Queue",
        body=json.dumps(task)
    )
    print(f"[ROVER {rover_id}] Published mine task to Demine-Queue: {task}")
    connection.close()

def main():
    if len(sys.argv) != 2:
        print("Usage: python client.py <rover_id>  # rover_id 1-10")
        sys.exit(1)

    rover_id = int(sys.argv[1])
    if rover_id < 1 or rover_id > 10:
        print("Rover ID must be 1-10")
        sys.exit(1)

    channel = grpc.insecure_channel("localhost:50051")
    stub = rover_pb2_grpc.RoverControlStub(channel)

    # 1. Get map
    print(f"[ROVER {rover_id}] Getting map...")
    map_resp = stub.GetMap(rover_pb2.MapRequest(rover_id=rover_id))
    ROWS, COLS, land = build_map_from_response(map_resp)
    land = copy.deepcopy(land)

    # Initial state
    x, y = 0, 0
    direction = "S"
    alive = True
    last_command = None

    print(f"[ROVER {rover_id}] Starting at ({x},{y}), facing {direction}")
```

```
    # 2. Get command stream (sequential processing, no threading)
    print(f"[ROVER {rover_id}] Getting commands...")
    cmd_stream = stub.GetCommands(rover_pb2.CommandRequest(rover_id=rover_id))

    for cmd_msg in cmd_stream:
        cmd = cmd_msg.cmd

        if not alive:
            break

        if cmd == "L":
            direction = turn_left(direction)
            print(f"[ROVER {rover_id}] Turn LEFT -> {direction}")

        elif cmd == "R":
            direction = turn_right(direction)
            print(f"[ROVER {rover_id}] Turn RIGHT -> {direction}")

        elif cmd == "D":
            # Rovers no longer dig in Lab 3 (optional: still clear if found)
            if land[x][y] == 1:
                land[x][y] = 0
                print(f"[ROVER {rover_id}] Digged current cell ({x},{y})")

        elif cmd == "M":
            dx, dy = MOVE[direction]
            nx, ny = x + dx, y + dy

            if nx < 0 or nx >= ROWS or ny < 0 or ny >= COLS:
                print(f"[ROVER {rover_id}] Boundary hit, staying at ({x},{y})")
                continue

            if land[nx][ny] == 1:
                print(f"[ROVER {rover_id}] Mine detected at ({nx},{ny})")

                # 3. Get mine serial via gRPC
                ms_req = rover_pb2.MineSerialRequest(rover_id=rover_id, row=nx,
col=ny)
                ms_resp = stub.GetMineSerial(ms_req)
                serial = ms_resp.serial
                print(f"[ROVER {rover_id}] Got serial: {serial}")

                # Lab 3: Publish to RabbitMQ Demine-Queue instead of disarming
                publish_mine_task(rover_id, nx, ny, serial)

                # Clear mine from rover's map view
                land[nx][ny] = 0

            x, y = nx, ny
```

```
            print(f"[ROVER {rover_id}] Moved to ({x},{y})")

        last_command = cmd

    print(f"[ROVER {rover_id}] FINISHED. Alive={alive}, pos=({x},{y}),
dir={direction}")

if __name__ == "__main__":
    main()
```

client.py

Part 4: Create the Deminers

The deminer program introduces two specialized workers (CLI args 1 or 2) that consume mine disarming tasks from RabbitMQ's Demine-Queue, compute security PINs using Lab 2's SHA256 brute-force algorithm, and publish results to Defused-Mines for ground control consumption. Unlike rovers, deminers implement fair workload distribution via prefetch_count=1, ensuring they process only one mine at a time to simulate "not busy" behavior as specified.

```
import sys
import pika
import json
import hashlib
import threading
import time


def compute_pin(serial: str) -> int:
    """
    Brute-force PIN like Lab 2 part2.py (6 leading zero SHA256).
    """
    pin = 0
    while True:
        temp_key = f"{pin}{serial}"
        h = hashlib.sha256(temp_key.encode()).hexdigest()
        if h.startswith("000000"):
            print(f"[DEMINER] Disarmed mine {serial} with PIN {pin}")
            return pin
        pin += 1


def deminer_worker(deminer_id: str):
    """
    Deminer subscribes to Demine-Queue, processes one mine at a time (if not
busy),
    publishes PIN to Defused-Mines.
    """
    defused_connection = None  # For publishing
```

```
    def callback(ch, method, properties, body):
        nonlocal defused_connection

        try:
            task = json.loads(body.decode())
            row = task["row"]
            col = task["col"]
            mine_id = task["mine_id"]
            serial = task["serial"]

            print(f"[DEMINER {deminer_id}] Assigned mine #{mine_id} at
({row},{col}), serial {serial}")

            # Simulate "demining time" (not busy during this)
            print(f"[DEMINER {deminer_id}] Demining... (computing PIN)")
            time.sleep(1)  # Brief delay to simulate work

            pin = compute_pin(serial)

            # Publish PIN to Defused-Mines for ground control
            if defused_connection is None or defused_connection.is_closed:
                defused_connection =
pika.BlockingConnection(pika.ConnectionParameters("localhost"))

            defused_channel = defused_connection.channel()
            defused_channel.queue_declare(queue="Defused-Mines")

            defused_channel.basic_publish(
                exchange="",
                routing_key="Defused-Mines",
                body=str(pin).encode()
            )
            print(f"[DEMINER {deminer_id}] Published PIN {pin} to Defused-Mines
queue")

            defused_channel.close()

            print(f"[DEMINER {deminer_id}] Completed mine #{mine_id}")
            ch.basic_ack(delivery_tag=method.delivery_tag)

        except Exception as e:
            print(f"[DEMINER {deminer_id}] Error processing task: {e}")
            ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)

    # Main connection to Demine-Queue
    connection = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
    channel = connection.channel()
    channel.queue_declare(queue="Demine-Queue")
```

```
    # Fair dispatch: only give 1 message at a time per deminer (simulates "not
busy")
    channel.basic_qos(prefetch_count=1)

    channel.basic_consume(
        queue="Demine-Queue",
        on_message_callback=callback,
        auto_ack=False  # Manual ack after successful processing
    )

    print(f"[DEMINER {deminer_id}] Started and listening to Demine-Queue
(prefetch=1)...")
    print(f"[DEMINER {deminer_id}] Ready for mine disarming tasks!")

    try:
        channel.start_consuming()
    except KeyboardInterrupt:
        print(f"[DEMINER {deminer_id}] Shutting down...")
        channel.stop_consuming()
        connection.close()
        if defused_connection and not defused_connection.is_closed:
            defused_connection.close()


if __name__ == "__main__":
    if len(sys.argv) != 2 or sys.argv[1] not in ["1", "2"]:
        print("Usage: python deminer.py [1|2]")
        sys.exit(1)

    deminer_id = sys.argv[1]
    deminer_worker(deminer_id)
```

deminer.py

Results

There were 5 terminal running simultaneously to produce an output. The first terminal was the RabbitMQ which was running successfully in the background. The other four terminals were opened in the lab 3 directory and ran from there. The server was ran in one terminal, the deminer 1 and 2 were ran in two separate terminals and the client for rovers 1-10 was ran in another terminal. RabbitMQ terminal was showing docker logs while it was running. The server started the gRPC server on port 50051 and printed any PINs published by deminers. The two deminers connected to RabbitMQ and subscribed to demine-queue. It then received the tasks from demine-queue, brute-forced valid PINs and published the PINs to defused-mines for ground control. Lastly, the client explored the map using the commands from the server terminal and published the mine tasks to RabbitMQ.

```
bob@comlab:~$ sudo docker run --name mqserver -p 5672:5672 rabbitmq
2026-03-13 01:57:59.805386+00:00 [notice] <0.45.0> Application syslog exited with reason: stopped
2026-03-13 01:57:59.808979+00:00 [notice] <0.209.0> Logging: switching to configured handler(s); fo
llowing messages may not be visible in this log output
2026-03-13 01:57:59.810067+00:00 [notice] <0.209.0> Logging: configured log handlers are now ACTIVE
2026-03-13 01:57:59.816432+00:00 [info] <0.209.0> ra: starting system coordination
2026-03-13 01:57:59.816554+00:00 [info] <0.209.0> starting Ra system: coordination in directory: /v
ar/lib/rabbitmq/mnesia/rabbit@560c2769ceed/coordination/rabbit@560c2769ceed
2026-03-13 01:57:59.823194+00:00 [info] <0.216.0> ra_coordination_log_ets: in system coordination i
nitialising. Mem table opts: [set,{write_concurrency,auto},public,{compressed,false}]
2026-03-13 01:57:59.865129+00:00 [info] <0.222.0> ra system 'coordination' running pre init for 0 r
egistered servers
2026-03-13 01:57:59.874038+00:00 [info] <0.223.0> ra: meta data store initialised for system coordi
nation. 0 record(s) recovered
2026-03-13 01:57:59.878065+00:00 [info] <0.226.0> segment_writer: upgrading segment file names to n
ew format in dirctory /var/lib/rabbitmq/mnesia/rabbit@560c2769ceed/coordination/rabbit@560c2769ceed
2026-03-13 01:57:59.906196+00:00 [notice] <0.228.0> WAL: ra_coordination_log_wal init, mem-tables t
able name: ra_coordination_log_open_mem_tables
2026-03-13 01:57:59.920857+00:00 [info] <0.209.0> ra: starting system quorum_queues
2026-03-13 01:57:59.920921+00:00 [info] <0.209.0> starting Ra system: quorum_queues in directory: /
var/lib/rabbitmq/mnesia/rabbit@560c2769ceed/quorum/rabbit@560c2769ceed
2026-03-13 01:57:59.921639+00:00 [info] <0.232.0> ra_log_ets: in system quorum_queues initialising.
 Mem table opts: [set,{write_concurrency,auto},public,{compressed,true}]
2026-03-13 01:57:59.922218+00:00 [info] <0.236.0> ra system 'quorum_queues' running pre init for 0
registered servers
2026-03-13 01:57:59.922952+00:00 [info] <0.237.0> ra: meta data store initialised for system quorum
_queues. 0 record(s) recovered
2026-03-13 01:57:59.923088+00:00 [info] <0.240.0> segment_writer: upgrading segment file names to n
```

RabbitMQ terminal

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab3/rover-api$ python3 server.py
RoverControl gRPC server running on port 50051...
[GROUND CONTROL] Waiting for defused mines on 'Defused-Mines' queue...
[GROUND CONTROL] Defused mine PIN received: 7614232
[GROUND CONTROL] Defused mine PIN received: 7614232
[GROUND CONTROL] Defused mine PIN received: 14603498
[GROUND CONTROL] Defused mine PIN received: 14603498
[GROUND CONTROL] Defused mine PIN received: 7614232
```

Server terminal

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab3/rover-api$ python3 deminer.py 1
[DEMINER 1] Started and listening to Demine-Queue (prefetch=1)...
[DEMINER 1] Ready for mine disarming tasks!
[DEMINER 1] Assigned mine #2 at (0,1), serial 4 3
[DEMINER 1] Demining... (computing PIN)
[DEMINER] Disarmed mine 4 3 with PIN 7614232
[DEMINER 1] Published PIN 7614232 to Defused-Mines queue
[DEMINER 1] Completed mine #2
[DEMINER 1] Assigned mine #9 at (2,0), serial 0 0 0
[DEMINER 1] Demining... (computing PIN)
[DEMINER] Disarmed mine 0 0 0 with PIN 14603498
[DEMINER 1] Published PIN 14603498 to Defused-Mines queue
[DEMINER 1] Completed mine #9
[DEMINER 1] Assigned mine #10 at (0,1), serial 4 3
[DEMINER 1] Demining... (computing PIN)
[DEMINER] Disarmed mine 4 3 with PIN 7614232
[DEMINER 1] Published PIN 7614232 to Defused-Mines queue
[DEMINER 1] Completed mine #10
```

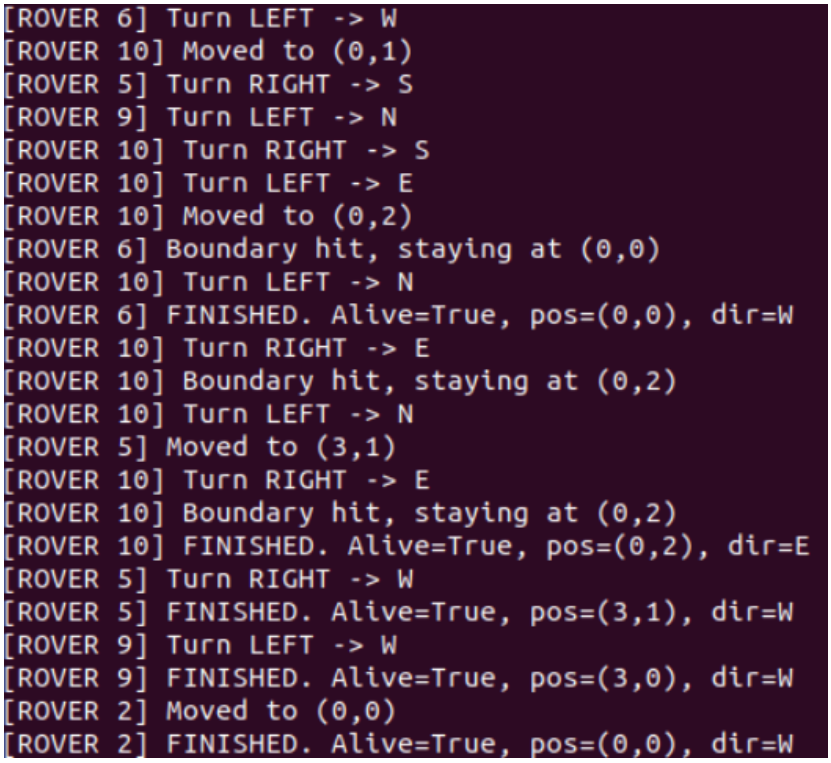
```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab3/rover-api$ python3 deminer.py 2
[DEMINER 2] Started and listening to Demine-Queue (prefetch=1)...
[DEMINER 2] Ready for mine disarming tasks!
[DEMINER 2] Assigned mine #6 at (0,1), serial 4 3
[DEMINER 2] Demining... (computing PIN)
[DEMINER] Disarmed mine 4 3 with PIN 7614232
[DEMINER 2] Published PIN 7614232 to Defused-Mines queue
[DEMINER 2] Completed mine #6
[DEMINER 2] Assigned mine #5 at (2,0), serial 0 0 0
[DEMINER 2] Demining... (computing PIN)
[DEMINER] Disarmed mine 0 0 0 with PIN 14603498
[DEMINER 2] Published PIN 14603498 to Defused-Mines queue
[DEMINER 2] Completed mine #5
```

2 deminer terminals

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab3/rover-api$ for i in {1..10}; do python3 client
.py $i & done
[11] 6156
[12] 6157
[13] 6158
[14] 6159
[15] 6160
[16] 6161
[17] 6162
[18] 6163
[19] 6164
[20] 6165
[1]   Exit 127               python client.py $i
[2]   Exit 127               python client.py $i
[3]   Exit 127               python client.py $i
[4]   Exit 127               python client.py $i
[5]   Exit 127               python client.py $i
[6]   Exit 127               python client.py $i
[7]   Exit 127               python client.py $i
[8]   Exit 127               python client.py $i
[9]   Exit 127               python client.py $i
[10]   Exit 127               python client.py $i
```

```
bob@comlab:~/Chanuth_Pathirana_COE892_Lab3/rover-api$ [ROVER 6] Getting map...
[ROVER 4] Getting map...
[ROVER 3] Getting map...
[ROVER 8] Getting map...
[ROVER 2] Getting map...
[ROVER 7] Getting map...
[ROVER 6] Starting at (0,0), facing S
[ROVER 6] Getting commands...
[ROVER 4] Starting at (0,0), facing S
[ROVER 4] Getting commands...
[ROVER 8] Starting at (0,0), facing S
[ROVER 8] Getting commands...
[ROVER 3] Starting at (0,0), facing S
[ROVER 3] Getting commands...
[ROVER 5] Getting map...
[ROVER 7] Starting at (0,0), facing S
[ROVER 7] Getting commands...
[ROVER 8] Turn RIGHT -> W
[ROVER 2] Starting at (0,0), facing S
[ROVER 2] Getting commands...
[ROVER 9] Getting map...
[ROVER 1] Getting map...
[ROVER 8] Boundary hit, staying at (0,0)
```

```
[ROVER 4] Turn RIGHT -> E
[ROVER 10] Got serial: 4 3
[ROVER 2] Published mine task to Demine-Queue: {'row': 0, 'col': 1, 'mine_id': 2, 'serial
': '4 3'}
[ROVER 4] Turn LEFT -> N
[ROVER 1] Turn LEFT -> W
[ROVER 4] FINISHED. Alive=True, pos=(0,0), dir=N
[ROVER 1] Boundary hit, staying at (1,0)
[ROVER 1] Turn RIGHT -> N
[ROVER 1] Turn LEFT -> W
[ROVER 1] Boundary hit, staying at (1,0)
[ROVER 1] FINISHED. Alive=True, pos=(1,0), dir=W
[ROVER 6] Published mine task to Demine-Queue: {'row': 0, 'col': 1, 'mine_id': 6, 'serial
': '4 3'}
[ROVER 9] Published mine task to Demine-Queue: {'row': 2, 'col': 0, 'mine_id': 9, 'serial
': '0 0 0'}
[ROVER 5] Published mine task to Demine-Queue: {'row': 2, 'col': 0, 'mine_id': 5, 'serial
': '0 0 0'}
[ROVER 9] Moved to (2,0)
[ROVER 5] Moved to (2,0)
[ROVER 9] Turn RIGHT -> W
[ROVER 2] Moved to (0,1)
[ROVER 5] Turn LEFT -> E
```



Client terminal

Conclusion

Ultimately, this lab provided a thorough understanding of how to transfer variables from one file to another, using prior knowledge of gRPC from Lab 2 and gaining newfound knowledge of RabbitMQ from this lab. Overall, this is a great learning experience in how cloud computing is crucial with communication that will be beneficial not just for Lab 4-5, but for future and revolutionary applications.